

aspgen Renoir Developer Guide

A practical, step-by-step guide to building and extending a Renoir-style DDD/Blazor application with aspgen.

A practical, step-by-step guide to building and extending a Renoir-style DDD/Blazor application with aspgen.

Version 1.0 | Architecture Theta | 04 August 2026

This guide walks through generating a real Renoir application from scratch, command by command, with the actual generated file trees and code. It then covers adding features to an existing app, contrasts Renoir's extension points against aspgen's other profiles, and closes with end-user tips and a copy-pasteable command cheat sheet. For the underlying architecture rationale see [doc/architecture-theta.md](#); for the full flag/profile matrix across all --app targets see [doc/aspgen-generation-decision-guide.md](#).

1. Architecture & patterns

Renoir (--app blazor) is aspgen's DDD/Blazor profile. It has no --simple/--backend ddd switch and no --database/--seed/--theme options — those apply only to webapi/wpf/fullstack. A Renoir project is always DDD-shaped and always targets SQL Server.

The vocabulary, in dependency order:

- **Bounded context** — a named grouping (add context Catalog) that becomes a folder under Contexts/ in each project. Contexts hold no code of their own; they exist to namespace the aggregates, value objects, services, repositories, and events that belong together.
- **Aggregate** — the consistency boundary and CRUD root (add aggregate Product ... --context Catalog). Generates a domain entity, an EF configuration, a CRUD service, and Blazor CRUD/detail pages.
- **Value object** — an immutable, identity-less concept (add value-object Money ... --context Catalog). Construction is the invariant boundary; there is no public setter.
- **Domain service** — a stateless policy that coordinates domain objects when a behavior doesn't belong naturally to one entity (add domain-service PricingPolicy --context Catalog).
- **Repository** — persistence indirection: the interface lives in DomainModel, the EF implementation lives in Persistence (add repository ProductRepository --aggregate Product --context Catalog).
- **Domain event** — an immutable record of a completed business fact (add event ProductCreated ... --context Catalog), free of infrastructure dependencies.

Dependency direction (arrows point "depends on"):

```
AppBlazor (Razor components, DI wiring)
|
v
Application (CRUD services, validators)
|
v
Infrastructure / Persistence (EF configurations, repositories, DbContext)
|
v
DomainModel (aggregates, value objects, domain services, events - no EF/HTTP dependency)
```

Other conventions worth knowing before generating anything:

- **BaseEntity + TimeStamp** — every aggregate inherits BaseEntity, giving it Id-independent soft delete (Deleted) and two TimeStamp complex properties (Created, LastUpdated) that track When/By and render via DateForDisplay()/AuthorForDisplay().

- **Partial-class split** — each aggregate is split across `{Aggregate}.cs` (constructor + properties, hand-editable region marked `// aspgen:navigation`) and `{Aggregate}.Methods.cs` (`Update`, `IsValid`, `Normalize`, `Import`, `ToString`). Regenerating fields only ever touches the first file plus the CRUD service/page — the `.Methods.cs` shape stays stable.
- **CommandResponse + TrySaveChangesAsync** — every mutating repository/service method returns `CommandResponse.Ok()/Fail().AddMessage(...)`, and `RenoirDemoDatabase.TrySaveChangesAsync` wraps `SaveChangesAsync` so callers never need a raw try/catch around EF calls.
- **IEntityTypeConfiguration<T> auto-discovery** — `RenoirDemoDatabase.OnModelCreating` calls `modelBuilder.ApplyConfigurationsFromAssembly(...)` once; each new aggregate's `{Aggregate}Configuration.cs` is picked up automatically, no manual registration.
- **// aspgen:services DI marker** — a single marker comment in `Program.cs` is where every add command that needs a DI registration (repository, CRUD service) inserts its `builder.Services.Add...` line. Never hand-edit around this marker; see section 5, Tips, tricks & patching.
- **RenoirSettings partial** — connection strings and app settings live in a hand-authored partial class `RenoirSettings` (`Application/Settings/RenoirSettings.cs`) rather than `appsettings.json`, so secrets can be swapped between Local/Live via `StorageDescriptor.Selector` without checking a real connection string into source control.

2. Build a Renoir app from scratch, step by step

Prerequisites: .NET SDK (see `global.json/README` for the pinned version) and a SQL Server instance — LocalDB is fine for development. Renoir has no `--database sqlite` option; the connection string always targets SQL Server via `RenoirSettings`. `aspgen` never runs `dotnet ef` migrations itself — you run EF tooling yourself once the domain shape is ready.

Step 1 — create the project

```
aspgen new RenoirDemo --app blazor --output ./RenoirDemo
```

This creates the full project skeleton plus `.aspgen/manifest.json`:

```
RenoirDemo/
.aspgen/manifest.json
RenoirDemo.sln
README.md
src/
RenoirDemo.AppBlazor/
App.razor, Routes.razor, Program.cs
Components/Layout/MainLayout.razor
Components/Pages/Home.razor
RenoirDemo.Application/
ApplicationServiceBase.cs
Settings/RenoirSettings.cs
RenoirDemo.DomainModel/
BaseEntity.cs, BaseEntity.Methods.cs
CommandResponse.cs, DomainException.cs, DomainGuard.cs, TimeStamp.cs
RenoirDemo.Infrastructure/
Security/IHashingService.cs
RenoirDemo.Persistence/
RenoirDemoDatabase.cs
RenoirDemo.Resources/
```

No `Contexts/` folder exists yet in any project — it's created the first time you add a context.

Step 2 — add a bounded context

```
aspgen add context Catalog --project ./RenoirDemo
```

This is a manifest-only operation; no files are written yet (a context is just a namespace grouping, populated when you add aggregates into it):

```
{
  "project": "RenoirDemo",
  "components": ["context:Catalog", "renoir"],
  "contexts": [{ "name": "Catalog" }]
}
```

Step 3 — add an aggregate

```
aspngen add aggregate Product name:string price:decimal active:bool --context Catalog --project
./RenoirDemo
```

New files (the CRUD-default shape — see --no-crud in section 3, Adding new features to an existing app, for the alternative):

```
src/RenoirDemo.AppBlazor/Components/Pages/Catalog/
ProductCrud.razor
ProductDetails.razor
src/RenoirDemo.Application/Contexts/Catalog/
ProductCrudService.cs
ProductValidator.cs
src/RenoirDemo.DomainModel/Contexts/Catalog/Aggregates/
Product.cs
Product.Methods.cs
src/RenoirDemo.Persistence/Contexts/Catalog/
ProductConfiguration.cs
ProductPersistence.cs
```

Product.cs — the hand-editable half of the aggregate:

```
public sealed partial class Product : BaseEntity
{
  private Product() { }

  public Product( string name, decimal price, bool active)
  {
    Name = DomainGuard.Required(name, nameof(name));
    Price = price;
    Active = active;
  }

  public long Id { get; private set; }
  public string Name { get; private set; } = default!;
  public decimal Price { get; private set; } = default!;
  public bool Active { get; private set; } = default!;
  // aspgen:navigation
}
```

ProductConfiguration.cs — picked up automatically by ApplyConfigurationsFromAssembly:

```
public sealed class ProductConfiguration : IEntityTypeConfiguration<Product>
{
  public void Configure(EntityTypeBuilder<Product> builder)
  {
    builder.HasKey(x => x.Id);
    builder.ComplexProperty(x => x.Created).IsRequired();
    builder.ComplexProperty(x => x.LastUpdated).IsRequired();
    builder.Property(x => x.Name).HasMaxLength(200);
  }
}
```

Step 4 — add a value object

```
aspngen add value-object Money amount:decimal currency:string --context Catalog --project
./RenoirDemo
```

New file: src/RenoirDemo.DomainModel/Contexts/Catalog/ValueObjects/Money.cs

```
public sealed record Money
{
  private Money( decimal amount, string currency)
  {
```

```

Amount = amount;
Currency = DomainGuard.Required(currency, nameof(currency));
}

public static Money Create( decimal amount, string currency) => new( amount, currency);
public decimal Amount { get; }
public string Currency { get; }
}

```

Value objects are generated standalone — wiring Money into Product (e.g. replacing the Price scalar) is a manual follow-up edit, by design; aspgen does not guess which aggregate should own a given value object.

Step 5 — add a domain service

```

aspgen add domain-service PricingPolicy --context Catalog --project ./RenoirDemo

```

New file: src/RenoirDemo.DomainModel/Contexts/Catalog/Services/PricingPolicy.cs

```

public sealed class PricingPolicy
{
    // Add ubiquitous-language operations here. Keep infrastructure concerns outside the domain.
}

```

This is intentionally an empty stub — a domain service's operations are specific to your business rules, so aspgen generates the placement and namespace only.

Step 6 — add a repository

```

aspgen add repository ProductRepository --aggregate Product --context Catalog --project
./RenoirDemo

```

New files — interface in the domain, implementation in persistence:

```

src/RenoirDemo.DomainModel/Contexts/Catalog/Aggregates/Repositories/IPProductRepository.cs
src/RenoirDemo.Persistence/Contexts/Catalog/Repositories/ProductRepository.cs

```

```

// Repository contract belongs to the domain; its implementation belongs to persistence.
public interface IPProductRepository
{
    Task<Product?> GetByIdAsync(long id, CancellationToken cancellationToken = default);
    Task<IReadOnlyList<Product>> GetAllAsync(CancellationToken cancellationToken = default);
    Task<CommandResponse> AddAsync(Product aggregate, CancellationToken cancellationToken =
default);
    Task<CommandResponse> SaveAsync(Product aggregate, CancellationToken cancellationToken =
default);
    Task<CommandResponse> DeleteAsync(long id, CancellationToken cancellationToken = default);
}

```

This also inserts a DI registration at the // aspgen:services marker in Program.cs:

```

// aspgen:services
builder.Services.AddScoped<IPProductRepository, ProductRepository>();
builder.Services.AddScoped<ProductCrudService>();

```

Step 7 — add a domain event

```

aspgen add event ProductCreated productId:long productName:string --context Catalog --project
./RenoirDemo

```

New file: src/RenoirDemo.DomainModel/Contexts/Catalog/Events/ProductCreated.cs

```

// A completed business fact. Keep this immutable and free of infrastructure dependencies.
public sealed record ProductCreated( long ProductId, string ProductName);

```

Like the domain service, the event record itself is generated — raising it from Product's methods and dispatching it is a manual step (Renoir does not ship a built-in event dispatcher/mediator by default).

Final combined tree

After all seven commands, the Catalog context looks like this end-to-end:

```
src/  
RenoirDemo.AppBlazor/Components/Pages/Catalog/  
ProductCrud.razor  
ProductDetails.razor  
RenoirDemo.Application/Contexts/Catalog/  
ProductCrudService.cs  
ProductValidator.cs  
RenoirDemo.DomainModel/Contexts/Catalog/  
Aggregates/  
Product.cs  
Product.Methods.cs  
Repositories/IProductRepository.cs  
Events/ProductCreated.cs  
Services/PricingPolicy.cs  
ValueObjects/Money.cs  
RenoirDemo.Persistence/Contexts/Catalog/  
ProductConfiguration.cs  
ProductPersistence.cs  
Repositories/ProductRepository.cs
```

Build and run

```
dotnet restore ./RenoirDemo/RenoirDemo.sln  
dotnet build ./RenoirDemo/RenoirDemo.sln  
dotnet run --project ./RenoirDemo/src/RenoirDemo.AppBlazor
```

The connection string lives in `RenoirDemo.Application/Settings/RenoirSettings.cs` (`SecretsSettings.Database.Local`, defaulting to a LocalDB connection string) — edit it directly, or point `StorageDescriptor.Selector` at "live" and fill in `.Live` for a real SQL Server instance. Once the schema looks right, run `dotnet ef migrations add Initial --project src/RenoirDemo.Persistence --startup-project src/RenoirDemo.AppBlazor` yourself — `aspngen` scaffolds the domain/persistence code but does not manage migrations.

3. Adding new features to an existing app

Adding a second bounded context or aggregate later

Nothing is regenerated from scratch — add commands only append. Adding a second context follows the exact same command shapes with new names:

```
aspngen add context Ordering --project ./RenoirDemo  
aspngen add aggregate Order customerName:string total:decimal --context Ordering --project  
./RenoirDemo
```

The manifest's contexts array simply grows a second entry; `Catalog` is untouched.

add entity-field — adding a property to an existing aggregate

```
aspngen add entity-field Product category:string --project ./RenoirDemo
```

This patches the aggregate, its CRUD service, and its CRUD page in place — no manual file editing needed, and it's safe to re-run for additional fields later. Before/after on the three touched files:

`Product.cs` (constructor + property added, `.Methods.cs`'s `Update/Import` also gained the new field):

```
- public Product( string name, decimal price, bool active)  
+ public Product( string name, decimal price, bool active, string category)  
{  
  Name = DomainGuard.Required(name, nameof(name));  
  Price = price;  
  Active = active;  
+  Category = DomainGuard.Required(category, nameof(category));  
}  
  
public long Id { get; private set; }
```

```
public string Name { get; private set; } = default!;
public decimal Price { get; private set; } = default!;
public bool Active { get; private set; } = default!;
+ public string Category { get; private set; } = default!;
```

ProductCrudService.cs (every projection/query that builds a ProductView or accepts a ProductRequest picks up the new column):

```
- public sealed record ProductRequest( string Name, decimal Price, bool Active);
- public sealed record ProductView(
- long Id, string Name, decimal Price, bool Active);
+ public sealed record ProductRequest( string Name, decimal Price, bool Active, string Category);
+ public sealed record ProductView(
+ long Id, string Name, decimal Price, bool Active, string Category);
```

ProductCrud.razor (a matching InputText bound into the edit form and a new grid column, using the same uiControl mapping recorded in the manifest):

```
<div class="mb-3">
<label>Active</label>
<InputCheckbox @bind-Value="form.Active" class="form-control" />
</div>
+ <div class="mb-3">
+ <label>Category</label>
+ <InputText @bind-Value="form.Category" class="form-control" />
+ </div>
```

--no-crud on add aggregate

Pass --no-crud when an aggregate is used only through domain services or other aggregates — never exposed as its own admin CRUD screen:

```
aspngen add aggregate LedgerEntry amount:decimal --context Accounting --no-crud --project
./RenoirDemo
```

This skips generating the CRUD service and Blazor CRUD/detail pages, giving you just the domain entity and EF configuration. You still need to run add repository yourself afterward if the aggregate needs direct persistence access — --no-crud only opts out of the CRUD/UI layer, not persistence.

4. Extending: module vs entity vs context vs different backend

aspngen's "add a building block" command depends entirely on which --app/backend profile the project was created with. There is no migration command between profiles — the choice is made once, at aspngen new time.

Profile (aspngen new flags)	Command to add a feature	What it generates
Renoir / Blazor (--app blazor)	add context then add aggregate/value-object/domain-service/repository/event	DDD building blocks scoped to a bounded context, as covered above
Simple Web API (--app webapi --simple / --app fullstack --simple)	add entity NAME prop:type...	EF model, DbSet, direct Minimal API CRUD — no context/aggregate concept
DDD Web API (--app webapi --backend ddd)	add entity NAME prop:type...	Domain entity, repository, CQRS CRUD, validation, endpoint (DDD-shaped, but flat — no bounded-context folders like Renoir)
WPF / fullstack desktop (--app wpf, --app fullstack)	add module NAME (new area) or add entity (adds to an existing module)	Prism module: views, ViewModel, navigation registration
Any webapi profile, desktop added later	add ui --project PATH	Adds the Prism/Dryloc Desktop project onto an existing server-only project, without regenerating the server

Switching between profiles (e.g. from Simple Web API to DDD, or from webapi to Renoir) is not supported as an in-place command — see [doc/aspngen-generation-decision-guide.md](#) for the full decision table if you're choosing a profile for a new project.

5. Tips, tricks & patching

- **Never hand-edit around `// aspngen:services`.** Every add command that needs a DI registration inserts its line directly above/below this marker in `Program.cs`. If you move or delete the marker comment, subsequent add commands will fail to find the insertion point.

- **The manifest is the source of truth, not the file tree.** `.aspngen/manifest.json` records every context, aggregate, and component flag (including theme/theme-mode for other profiles). add commands read it to validate preconditions (e.g. add aggregate requires the context to already exist in the manifest) — don't hand-edit generated files in ways that make them inconsistent with what the manifest thinks exists.

- **add commands are idempotent-looking and safe to re-run** for incremental changes like add entity-field — they patch specific, well-known insertion points rather than regenerating whole files, so re-running with a new field is the norm, not a hack.

- **Dotted project names work end-to-end.** `aspngen new Markosoft.Commerce --app blazor` produces a consistent `Markosoft.Commerce` namespace, `.csproj` filenames, project references, and `.sln` entries — no special-casing needed for multi-segment names.

- **Corporate NuGet restore workaround.** If `dotnet restore/dotnet build` fails with NU1301 "No such host is known" behind a corporate proxy or VPN-only NuGet source, restore explicitly from `nuget.org` first, then build normally:

```
``bash dotnet restore ./RenoirDemo/RenoirDemo.sln -s https://api.nuget.org/v3/index.json dotnet build
./RenoirDemo/RenoirDemo.sln ``
```

- **Use add entity-field for properties you forgot at aggregate-creation time** rather than hand-editing `Product.cs/ProductCrudService.cs/ProductCrud.razor` — it keeps the three files' shapes in sync and handles the `uiControl` mapping for you.

- **--no-crud is a one-way choice at creation time** — there's no add-based way to retroactively add the CRUD service/pages to an aggregate created with `--no-crud`; recreate it without the flag, or hand-author the CRUD service/pages by copying the shape from another aggregate in the same project.

- **Troubleshooting a build failure right after add:** check the manifest first — confirm the context/aggregate you referenced actually exists there, and confirm the `// aspngen:services` marker in `Program.cs` wasn't accidentally removed or duplicated. Most post-add compile errors trace back to one of those two things rather than a bug in the generated code itself.

6. Appendix: command cheat sheet

```
# Create the project
aspngen new RenoirDemo --app blazor --output ./RenoirDemo

# Add a bounded context, then its building blocks
aspngen add context Catalog --project ./RenoirDemo
aspngen add aggregate Product name:string price:decimal active:bool --context Catalog --project
./RenoirDemo
aspngen add value-object Money amount:decimal currency:string --context Catalog --project
./RenoirDemo
aspngen add domain-service PricingPolicy --context Catalog --project ./RenoirDemo
aspngen add repository ProductRepository --aggregate Product --context Catalog --project
./RenoirDemo
aspngen add event ProductCreated productId:long productName:string --context Catalog --project
./RenoirDemo

# Add a field to an existing aggregate later
aspngen add entity-field Product category:string --project ./RenoirDemo
```

```
# CRUD-less aggregate (domain-only, no admin screen)
aspngen add aggregate LedgerEntry amount:decimal --context Accounting --no-crud --project
./RenoirDemo

# Build and run
dotnet restore ./RenoirDemo/RenoirDemo.sln
dotnet build ./RenoirDemo/RenoirDemo.sln
dotnet run --project ./RenoirDemo/src/RenoirDemo.AppBlazor

# Corporate NuGet restore workaround
dotnet restore ./RenoirDemo/RenoirDemo.sln -s https://api.nuget.org/v3/index.json
```